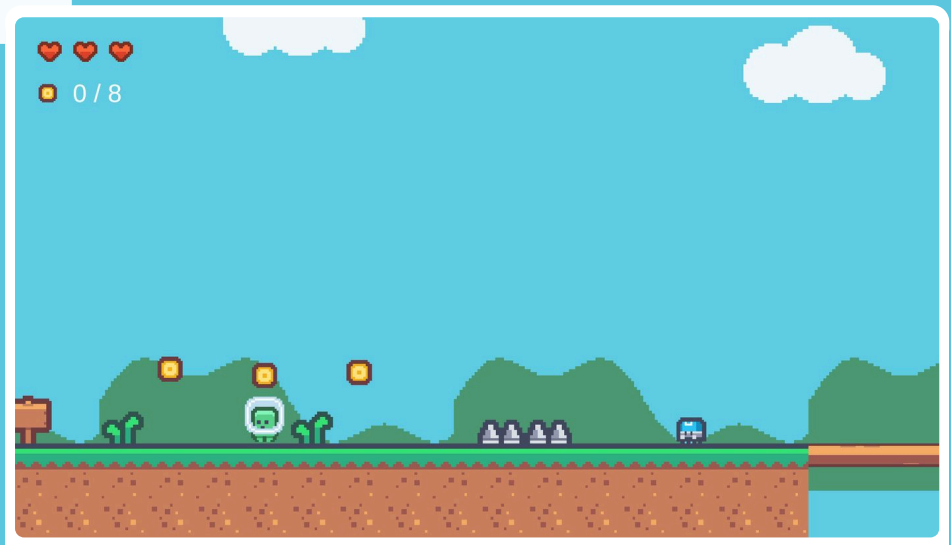
A no-code Unity starter kit for game jams

JamKit

Drag a prefab. Set its fields. Press Play.

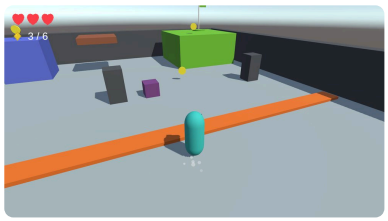
Platformer, top-down and third-person games, built without writing code.



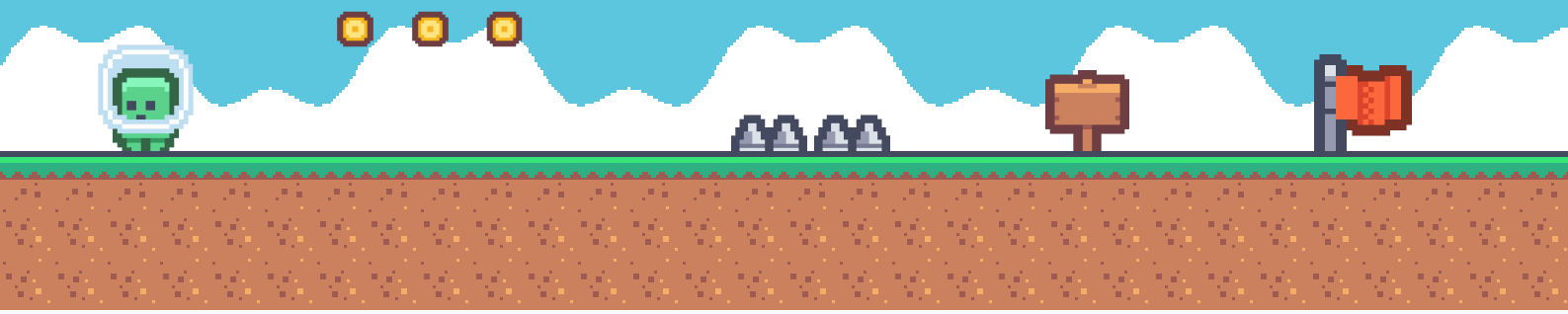
2D platformer



2D top-down



3D third-person



GameJamPlus Bangladesh 2026 pre-jam workshop

23 October 2026, online. Organized by Battery Low Interactive, Official Local Organizer of GameJamPlus Bangladesh.

Mentors: Stacey Kalkowski (2D art and Unity), Rakib Jahan (game design and scoping), Shounak Sobahani (no-code implementation).

JamKit v1.0 by **Shounak Sobahani**, Product Architect and Engineering Lead at Talent Pro. Unity 6000.3.14f1.

Scan to download JamKit

1 Quick start



Install (once, before the jam)

- [ ] **Unity Hub** — unity.com/download (free Personal licence).
- [ ] **Unity 6000.3.14f1** (Unity 6.3 LTS) with **Web Build Support** and **Windows Build Support** (Mac: Mac Build Support). One-click link for the Hub: `unityhub://6000.3.14f1/d68c3f99a318`
- [ ] About **10 GB** free disk space.
- [ ] Download `JamKit_Template_v1.0.zip` from the JamKit Google Drive folder (or github.com/shounak00/JamKit → Releases). Unzip it.
- [ ] Unity Hub → **Add** → Add project from disk → the `JamKit` folder. Open it once: the first import takes a few minutes.
- [ ] Open a demo scene in `JamKit/Scenes/Demo` and press **Play** to check everything works.
- [ ] A free itch.io account (that's where your game goes).

Already have a project? Import `JamKit_v1.0.unitypackage` and set Project Settings → Player → Active Input Handling = **Both**. The startup notice "This project uses Input Manager, which is marked for deprecation" is harmless: ignore it.

The golden rule

Drag → Set → Play

Drag a prefab from the Project window into your level · set its fields in the Inspector (hover any field for a tooltip) · press Play. No code.

Your first level in 6 steps

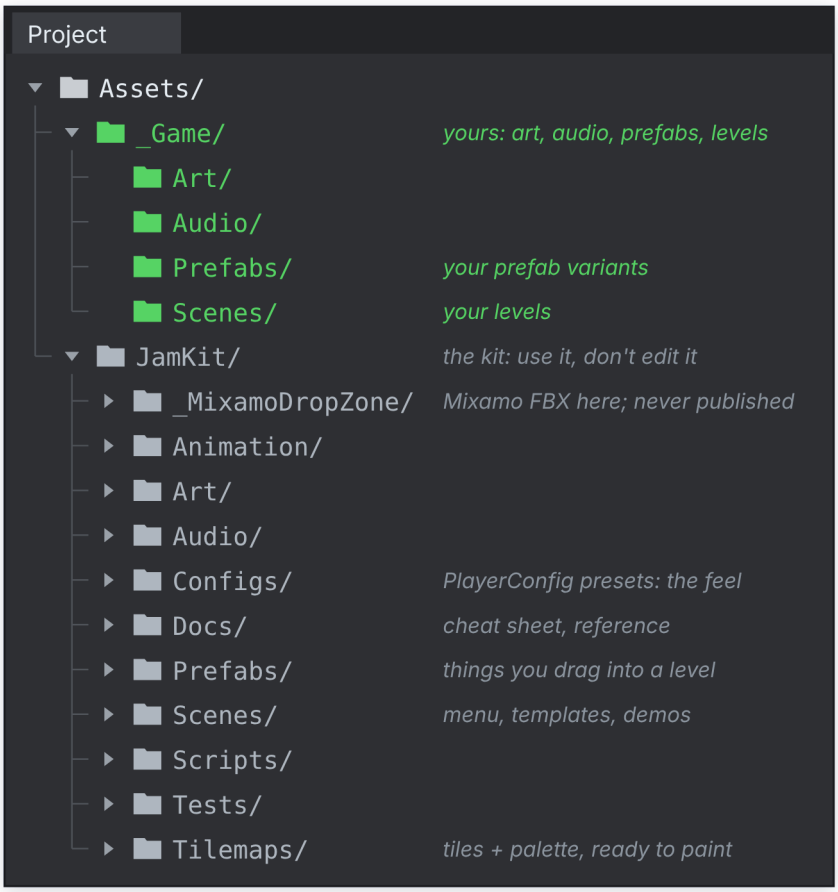
1. `JamKit → New Level → 2D Platformer` (or 2D Top-Down / 3D Third-Person). It's saved in `Assets/_Game/Scenes` and added to Build Settings.
2. Paint ground: Window → 2D → Tile Palette → `JamKit_Palette`, paint on `Tilemap_Ground`.
3. Drag `Coin_2D` and `Hazard_2D` from `JamKit/Prefabs/World` into the level. `Goal_2D` is already there.
4. Press Play. Touch the flag: you win. Touch the hazard 3 times: you lose. Press **R** to restart.
5. `JamKit → Validate Scene` → fix anything red (click **Fix**).
6. `JamKit → Prepare WebGL Build` → build → zip the folder's **contents** → upload to itch.io.

The `JamKit` menu



New Level	new level from a template
3D	Mixamo setup, fit the 3D collider
Validate Scene	✅/❌ checklist with Fix buttons
Prepare WebGL Build	itch.io-ready web build
Help (Cheat Sheet)	opens this PDF
Welcome Window	the first-run welcome

2 Asset hierarchy & where things go



Yours (Assets/_Game/) Kit (Assets/JamKit/) Swap point

System objects

`_Game/` is yours. Your art, audio, prefab variants and levels go here (Art, Audio, Prefabs, Scenes). Kit updates never touch it.

`JamKit/` is the kit. Use it, don't edit it: a kit update (v1.0.1) can replace these files and your changes would be lost.

Change a kit prefab: make a variant

1. Right-click the kit prefab, e.g. `JamKit/Prefabs/World/Coin_2D`.
2. **Create → Prefab Variant.**
3. Move the new `Coin_2D Variant` into `_Game/Prefabs` and edit that.

Where to look

`Prefabs/`	things you drag into a level (Player, World, Systems, UI)
`Scenes/`	`MainMenu`, level templates, `Demo/`
`Configs/`	`PlayerConfig` presets: the feel
`Tilemaps/`	tiles + palette, ready to paint
`Docs/`	this cheat sheet, `JamKit_Reference.md`, README
`_MixamoDropZone/`	empty: drop Mixamo FBX here; git-ignored, never published

Naming conventions

Player prefabs	Enemy prefabs	Levels	Sprites	Audio	Materials
`Player_*`	`Enemy_*`	`Level_##_Name`	`spr_name_action_##`	`sfx_*` / `mus_*`	`mat_*`
`Player_Knight`	`Enemy_Slime`	`Level_01_Forest`	`spr_knight_run_01`	`sfx_jump`	`mat_lava`

3 Scene hierarchy

Every level has the same sections. Put new things under the right header. The headers are plain empty GameObjects that keep things tidy.

Hierarchy · 2D

```
Level_Template_Platformer2D
  ---- SYSTEMS ----
    GameManager
    MusicPlayer
    EventSystem
  ---- CAMERA ----
    Main Camera
  ---- PLAYER ----
    Player
    SpawnPoint
  ---- LEVEL ----
    Grid
      Tilemap_Ground
      Tilemap_Decor
      Tilemap_Background
    Hazards
    Collectibles
    Enemies
    Platforms
    Triggers & Doors
    Goal
      Goal_2D
  ---- BACKGROUND ----
  ---- UI ----
    UI_Canvas
```

Hierarchy · 3D

```
Level_Template_ThirdPerson3D
  ---- SYSTEMS ----
    GameManager
    MusicPlayer
    EventSystem
  ---- CAMERA ----
    Main Camera
  ---- PLAYER ----
    Player
    SpawnPoint
  ---- LEVEL ----
    Environment
      Floor
      Walls
      Props
      Lighting
    Hazards
    Collectibles
    Enemies
    Platforms
    Triggers & Doors
    Goal
      Goal_3D
  ---- BACKGROUND ----
    Skybox_Note (…)
  ---- UI ----
    UI_Canvas
```

Yours (Assets/_Game/) · Kit (Assets/JamKit/) · Swap point · System objects

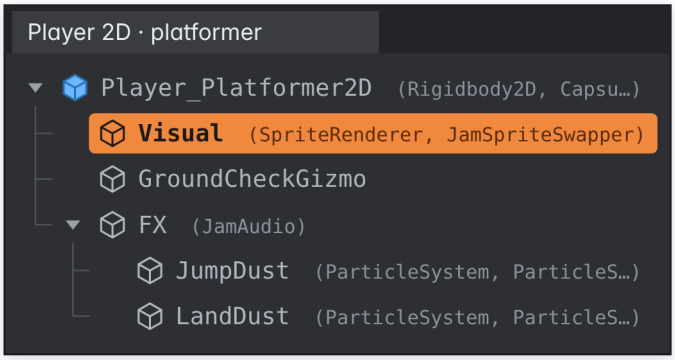
`---- SYSTEMS ----`	`GameManager` (the referee: win / lose / pause, lives), `MusicPlayer`, `EventSystem` (makes buttons clickable). Leave them alone.
`---- CAMERA ----`	`Main Camera`: follows the player (2D) or orbits it (3D). URP renderer 0 = 2D, 1 = 3D.
`---- PLAYER ----`	`Player` and its `SpawnPoint`.
`---- LEVEL ----`	Your level. 2D: paint on `Tilemap_Ground` (it has the colliders), decorate on `Tilemap_Decor` / `Tilemap_Background`. 3D: block out under `Environment` (every piece needs a collider). Drop prefabs into `Hazards`, `Collectibles`, `Enemies`, `Platforms`, `Triggers & Doors`, `Goal`.
`---- BACKGROUND ----`	2D: parallax layers (`JamParallax`). 3D: set the skybox in Window → Rendering → Lighting.
`---- UI ----`	`UI_Canvas`: `HUD`, `PauseMenu`, `WinScreen`, `LoseScreen`. Already wired; just restyle.

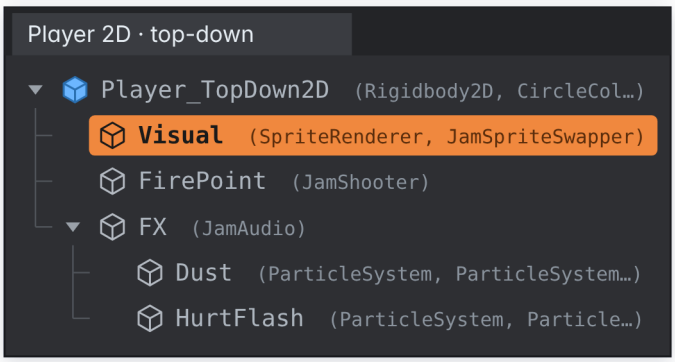
Build Settings order

File → Build Profiles → Scene List: `MainMenu` must be **scene 0**, then your levels in play order. `JamKit → New Level` adds new levels for you; `JamKit → Validate Scene` → Fix adds a missing one.

4 Prefab anatomy

Never edit the root. Swap or restyle `Visual` (2D) or `Model` (3D). Every World prefab is **Root / Visual / FX**.

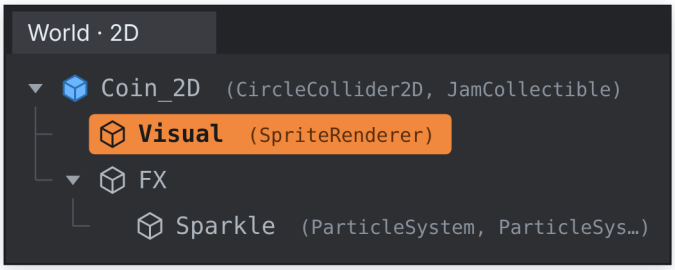


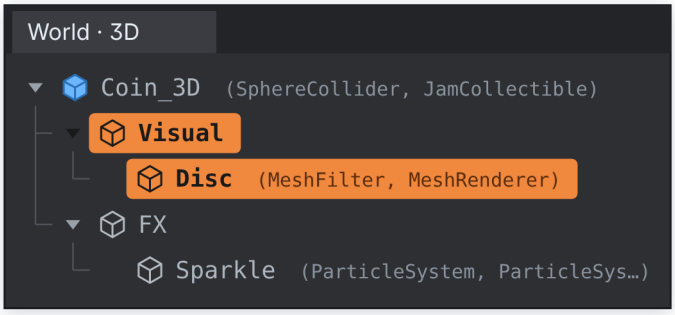


`JamShooter` sits on the top-down player's `FirePoint`. `FX` holds particles and a `JamAudio` for the jump sound.

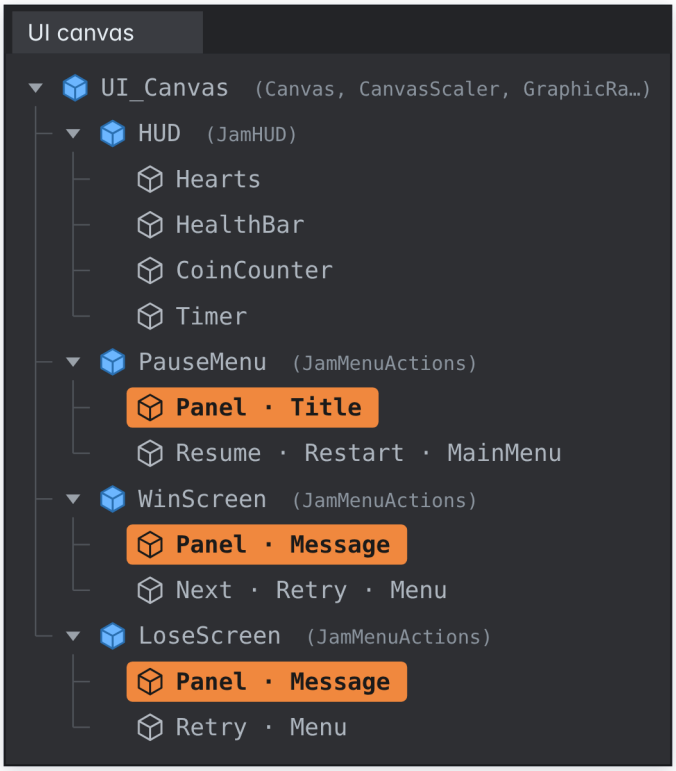


`Model` holds the `Animator`: put your humanoid character under it (the kit ships a teal capsule, `Placeholder_Body` + `Placeholder_Visor`). Its **Humanoid Avatar** maps the character's skeleton bones, so any humanoid works with the kit's animations. `CameraTarget` is the shoulder-height point the camera orbits.





Root = collider + behaviour · `Visual` = the art · `FX` = particles. Every World prefab comes as `_2D` and `_3D` with the same components; only the collider differs.



Change text: select `Title` (PauseMenu) or `Message` (WinScreen / LoseScreen). Buttons call `JamMenuActions`; button labels are each button's `Text` child. HUD parts hide themselves when unused.

Yours (Assets/_Game/) Kit (Assets/JamKit/) Swap point System objects

5 Component reference

Every component is in **Add Component**: type `Jam`. Each shows a one-line help box at the top of its Inspector. Full field lists: `Docs/JamKit_Reference.md`.

Systems	
`JamGameManager`	The referee: playing / paused / won / lost, shows the right screen, Restart / Next Level / Menu. One per level. Lives mode (useLives, lives) lives here.
`JamCameraFollow2D`	Smooth 2D follow with look-ahead; Use Bounds stops it at the level edges.
`JamThirdPersonCamera`	3D orbit camera: mouse / right stick, pulls in at walls, locks the cursor (Esc frees it).
`JamMusicPlayer`	Background music that keeps playing between levels.
`JamAudio`	Plays sounds from any event: JamAudio → PlayOneShot, drop your sound in.
`JamSceneLoader`	Scene changes for buttons/events: LoadScene(name), LoadNext(), Reload(), Quit().
Player	
`JamPlayerPlatformer2D`	Side-view run & jump. Feel lives in the PlayerConfig; swap art on Visual.
`JamPlayerTopDown2D`	8-direction movement, optional dash. Pairs with JamShooter and JamChaser.
`JamPlayerThirdPerson3D`	Camera-relative 3D movement, Shift to run. Swap the character on Model.
`JamHealth`	Hit points (hearts). At zero: player loses (or loses a life), enemies die.
`JamSpriteSwapper`	Artist-friendly animation: drag frames into Idle, Run, Jump... on Visual.
`JamHumanoidAnimator`	Feeds movement into the humanoid Animator. Root motion stays off.
World	
`JamCollectible`	Pickup: adds Amount to the counter named Item Id (coin, key, any word).
`JamDamageZone`	Hurts what touches it: spikes, lava, an enemy body. 2D and 3D.
`JamGoal`	Touch to win. Can be locked until you have enough of an item.
`JamCheckpoint`	Respawn point after falling or losing a life.
`JamTriggerZone`	The universal glue: player walks in (or presses E) → run events. Can need an item.
`JamDoor`	Open / Close / Toggle from any event; collider off when open.
`JamMover`	Moves between Waypoints children: platforms, elevators. Carries the player.
`JamPatrolEnemy`	Walks back and forth, turns at walls and ledges; stomp it from above.
`JamChaser`	Chases the player within range (top-down enemies).
`JamShooter`	Fires a projectile on a timer (turrets) or on Fire (J / Ctrl / left mouse).
`JamProjectile`	Bullet fired by JamShooter: flies straight, hurts what it hits.
`JamSpawner`	Spawns a prefab every few seconds, up to a max alive.
`JamTimer`	Level timer: count up, or count down to Lose (race) or Win (survive).
`JamParallax`	Background layer that scrolls slower than the camera.
`JamDialogueTrigger`	Signs, tutorials, story: a text box when the player walks in (or presses E).
UI	
`JamHUD`	Hearts (or a bar), item counter (coins by default), timer. Unused parts hide.
`JamMenuActions`	Button actions: Resume, Restart, Next Level, Main Menu, Quit. Already wired.
Static helpers (nothing to add)	
`JamInput`	Static helper: the one place input is read. Nothing to add to a scene.
`JamInventory`	Static counters by item id. Filled by JamCollectible, read by HUD, Goal, Trigger Zone.

Sounds from any event: drag an object with `JamAudio` into the event slot → `JamAudio → PlayOneShot` → drop your sound. Player events: `OnJump`, `OnLand`, `OnDash`, `OnFootstep`, `OnHurt`, `OnDeath`.

Recipes 1–5

1 Collect all coins to unlock the goal

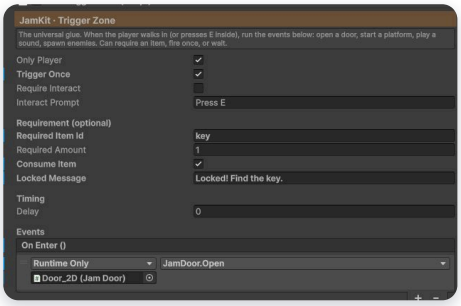
1. Drag `Coin_2D` into `Collectibles`; Ctrl+D to duplicate.
2. Select `Goal_2D` (under `Goal`).
3. `JamGoal`: **Required Item Id** = `coin`.
4. Tick **Require All In Level** (or set **Required Amount**, e.g. 10).
5. Optional: edit **Locked Message**; `{have}` / `{need}` become numbers.
6. Play: the goal shows the hint until you have them all.

Goal_2D · JamGoal

Required Item Id `requiredItemId`	coin
Require All In Level `requireAllInLevel`	✓
Locked Message `lockedMessage`	Collect {need} {item} first! ({have}/{need})

2 Key opens a door

1. Drag `Key_2D`, `Door_2D` and a `TriggerZone_2D` in front of the door.
2. Zone: **Required Item Id** = `key`, tick **Consume Item**.
3. **On Enter ()** → **+** → drag `Door_2D` into the slot.
4. Pick `JamDoor → Open`.
5. Optional: set the door's **Open Sound**.
6. Play: no key → "You need a key." With the key → the door opens.



`JamTriggerZone` → `JamDoor.Open`

3 Moving platform / elevator

1. Drag `MovingPlatform_2D` into `Platforms`.
2. Expand it → `Waypoints` → move `Point1` in the Scene view. The start position is the first point.
3. More stops: duplicate `Point1` (Ctrl+D).
4. `JamMover`: **Speed**, **Mode** (PingPong / Loop / Once), **Wait At Point**.
5. Elevator: tick **Start On Trigger**: it waits until the player stands on it, or a Trigger Zone calls `JamMover → StartMoving`.

MovingPlatform_2D · JamMover

Speed `speed`	2
Mode `mode`	PingPong
Wait At Point `waitAtPoint`	0.5
Start On Trigger `startOnTrigger`	☐

The player rides it automatically.

4 Enemy you can stomp

1. Drag `PatrolEnemy_2D` into `Enemies`.
2. `JamPatrolEnemy`: keep **Stompable** ticked; **Bounce Force** = how high you bounce.
3. **Patrol Range**: turns after this many units (0 = only at walls and ledges). **Turn At Ledges** keeps it on its platform.
4. Its body hurts through the `JamDamageZone` on the same object (**Damage** 1).
5. Optional: a sound on `OnStomped`.
6. Play: land on its head to defeat it; touching its side costs a heart.

PatrolEnemy_2D · JamPatrolEnemy

Stompable `stompable`	✓
Bounce Force `bounceForce`	14
Patrol Range `patrolRange`	3
Turn At Ledges `turnAtLedges`	✓

5 Survive 60 seconds (timer win)

1. Select `GameManager` → Add Component → `JamTimer`.
2. **Mode** = CountDown, **Seconds** = 60.
3. **On Zero** = Win.
4. The HUD timer appears by itself.
5. Make surviving hard: `Spawner_2D`, `Chaser_2D`, hazards. A win Timer counts as a win condition for Validate Scene.

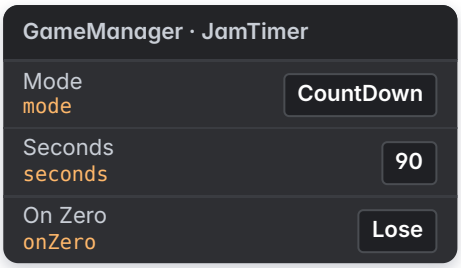
GameManager · JamTimer

Mode `mode`	CountDown
Seconds `seconds`	60
On Zero `onZero`	Win

7 Recipes 6–10

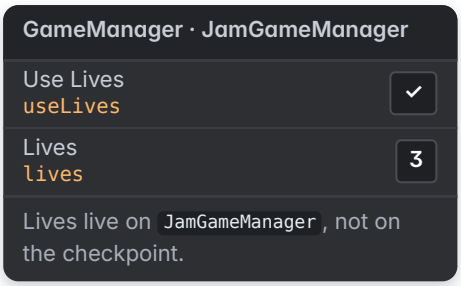
6 Race the clock (timer lose)

1. Select `GameManager` → Add Component → `JamTimer`.
2. **Mode** = CountDown, **Seconds** = e.g. 90.
3. **On Zero** = Lose.
4. Keep `Goal_2D`: reach it before zero to win.
5. Bonus time: a `TriggerZone_2D` with **Trigger Once**, **On Enter** → `JamTimer → AddTime` (e.g. 10).



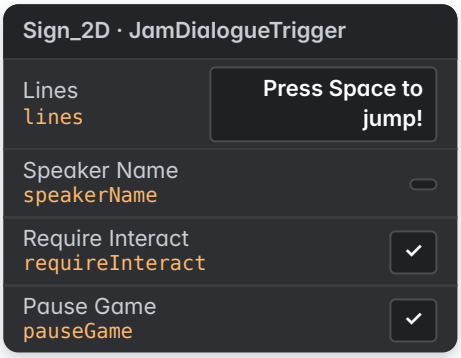
7 Checkpoints and lives

1. Drag `Checkpoint_2D` along the level.
2. Select `GameManager`: tick **Use Lives**, set **Lives** (default 3).
3. Dying now costs a life and respawns you at the last checkpoint; at 0 lives the Lose screen shows.
4. Falling below the player's **Kill Y** costs 1 heart and respawns you.
5. Optional: **Heal On Activate** on the checkpoint.



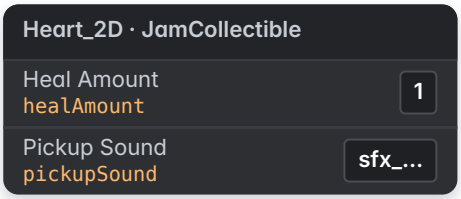
8 Signpost / tutorial text

1. Drag `Sign_2D` into the level.
2. `JamDialogueTrigger` → **Lines**: type one line per entry (+ adds more).
3. Leave **Speaker Name** empty for a sign; fill it for a character.
4. Tick **Require Interact** to wait for **E**; **Trigger Once** shows it only once.
5. **Pause Game** is on by default. Space / E advances.



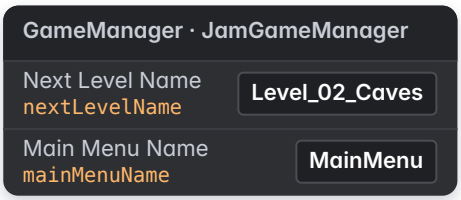
9 Health pickup

1. Drag `Heart_2D` into `Collectibles`.
2. `JamCollectible` → **Heal Amount** = hearts restored (0 = none).
3. Maximum hearts: the player's `JamHealth` → **Max Health** (default 3).
4. Play: get hurt, grab the heart, watch the HUD refill.



10 Multiple levels (Next Level flow)

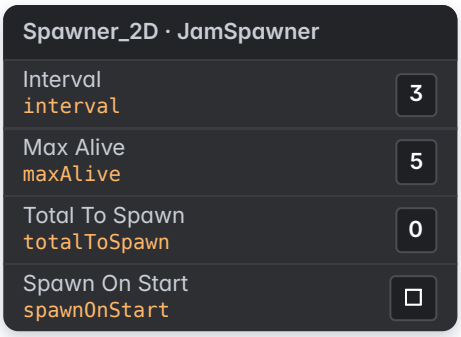
1. `JamKit → New Level` for each level (saved in `Assets/_Game/Scenes`, added to Build Settings).
2. In level 1 select `GameManager` → **Next Level Name** = the exact name of level 2.
3. The Win screen always shows; its **Next** button loads that scene (blank = button hidden).
4. Build Settings: `MainMenu` = 0, then levels in order.
5. Main menu **Play** loads `JamMenuActions` **First Level Name** (blank = next scene in Build Settings).
6. Validate Scene → Fix adds any missing scene.



8 Recipes 11–14

11 Spawner wave

1. Drag `Spawner_2D`; set **Prefab** (e.g. `PatrolEnemy_2D`).
2. **Interval**, **Max Alive**, **Total To Spawn** (0 = forever).
3. Spawn points are the children of `SpawnPoints`: move or duplicate them.
4. Start on cue: untick **Spawn On Start**; a Trigger Zone's **On Enter** → `JamSpawner → StartSpawning`.
5. **Random Radius** spreads them out.



12 Top-down: shoot projectiles at chasers

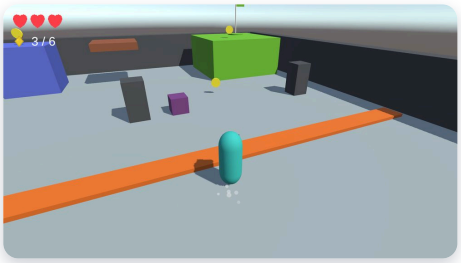
1. `JamKit → New Level → 2D Top-Down`. The player already shoots: `JamShooter` on its `FirePoint`.
2. Fire: **J**, **Left Ctrl** or **left mouse**; it aims where you face.
3. Drag `Chaser_2D` into `Enemies`: **Speed**, **Detect Range**.
4. Bullets are `Projectile_2D` (**Affects** = Enemies); chasers have `JamHealth`.
5. Turrets: `Turret_2D` fires `EnemyProjectile_2D`; tick **Aim At Player**.



`Demo_2D_TopDown`

13 3D: collect & reach the goal

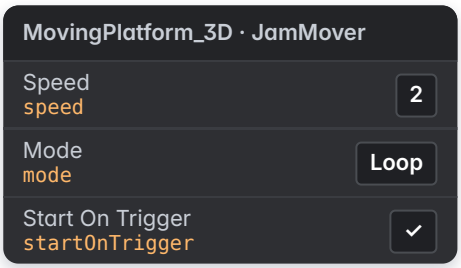
1. `JamKit → New Level → 3D Third-Person`.
2. Block out under `Environment`: GameObject → 3D Object → Cube (cubes come with a Box Collider).
3. Drag `Coin_3D` into `Collectibles`; `Goal_3D` is already under `Goal`.
4. `Goal_3D`: **Required Item Id** = `coin`, tick **Require All In Level**.
5. Play: WASD (camera-relative), mouse to look, Shift to run, Space to jump, Esc frees the cursor.



`Demo_3D_ThirdPerson`

14 3D: moving platform the player rides

1. Drag `MovingPlatform_3D` into `Platforms`.
2. Move the child of `Waypoints` in the Scene view (the start position is the first point).
3. `JamMover`: **Speed**, **Mode**, **Wait At Point**.
4. The 3D player inherits the platform's movement: just stand on it.
5. Elevator: tick **Start On Trigger**.



The glue for everything else: `TriggerZone_2D` / `TriggerZone_3D`. In its **On Enter** list, call any function: `JamDoor → Open`, `JamMover → StartMoving`, `JamSpawner → StartSpawning`, `JamAudio → PlayOneShot`, `JamTimer → AddTime`, `JamGameManager → Win`.

9 Player controllers & 3D quick start



`JamPlayerPlatformer2D`

Choose this if your game is side-view: run, jump, platforms, stomping enemies.

`Player_Platformer2D` · default `PlayerConfig_Snappy`

`JamPlayerTopDown2D`

Choose this if you look down on the world: 8 directions, dash, shooting, chasers.

`Player_TopDown2D` · `PlayerConfig_TopDown`

`JamPlayerThirdPerson3D`

Choose this if you want 3D: camera-relative movement, humanoid character.

`Player_ThirdPerson3D` · `PlayerConfig_ThirdPerson_Default`

Input map

Action	Keyboard / mouse	Gamepad
Move	WASD / Arrows	Left stick / D-pad
Jump	Space (W / Up also jump in the 2D platformer)	South (A / ×)
Sprint / Dash	Left Shift	West (X / □) or Right trigger
Interact	E	North (Y / △)
Fire	J, Left Ctrl or Left mouse	East (B / ○)
Look / Camera	Mouse	Right stick
Pause	Esc or P	Start
Restart	R	Select
Advance dialogue	Space, E, Fire or Enter	South / North / East

Feel: `PlayerConfig`

Drag a preset into the player's **Config** field. Edits made while playing are kept. **Reset to defaults** undoes them. New: Create → JamKit → Player Config.

Platformer 2D · presets `PlayerConfig_Snappy` (default), `PlayerConfig_Floaty`, `PlayerConfig_Heavy`
tweak first: `moveSpeed`, `jumpHeight`, `gravityScale`, `fallGravityMultiplier`, `maxJumps`

Top-Down 2D · preset `PlayerConfig_TopDown`
tweak first: `moveSpeed`, `acceleration`, `canDash`, `dashSpeed`, `facing`

Third Person 3D · presets `PlayerConfig_ThirdPerson_Default`, `PlayerConfig_ThirdPerson_Platformer`, `PlayerConfig_ThirdPerson_Explorer`
tweak first: `moveSpeed`, `runSpeed`, `jumpHeight`, `turnSmoothTime`, `lookSensitivity`

`maxJumps` 2 = double jump · 0 = no jumping.

3D quick start

1. `JamKit → New Level → 3D Third-Person`.
2. Block out with cubes (or ProBuilder) under `Environment`. Everything you stand on needs a collider.
3. Use the `_3D` prefabs: `Coin_3D`, `Hazard_3D`, `Goal_3D`, `TriggerZone_3D` …

Mixamo in 3 steps

1. mixamo.com (free Adobe account) → Characters → Download: **FBX for Unity (.fbx)**, **T-pose**, **With Skin**. Animations (optional): Idle, Walking, Running, Jump, Falling Idle, Landing → **In Place**, FBX for Unity, **Without Skin**.
2. Drop every FBX into `JamKit/_MixamoDropZone`.
3. `JamKit → 3D → Setup Mixamo Character` → read the ✅/❌ report. Your player is `Player_ThirdPerson3D_<Name>` in `_MixamoDropZone/Generated/`; **Replace player in current scene** swaps it in.

Tick "In Place" on every animation. Root motion is off: the controller moves, the animation only shows. Without In Place the character slides and snaps back.

Licence: Mixamo characters and animations are fine inside your game (credit them). Never re-upload the raw FBX files anywhere: the drop zone is git-ignored for that reason.

Animator parameters

`JamHumanoid.controller`, shared by every humanoid.

Speed	Grounded	VerticalVelocity	Jump	Hurt	Dead	Victory
Float	Bool	Float	Trigger	Trigger	Bool	Trigger

States: Locomotion (blend on `Speed`: Idle 0 · Walk 1 · Run 2), Jump, Fall, Land, Hurt, Death, Victory. Own clips: Setup Mixamo Character maps them by name into an override controller.

10 Art & audio

Sprite import settings

Texture Type	Sprite (2D and UI)
Pixels Per Unit	**your tile size**, so 1 tile = 1 unit (any pixel size works; the kit's Kenney art uses 18 px tiles)
Pivot	**Bottom Center** for characters; Center for the rest
Filter Mode	**Point (no filter)** for pixel art
Compression	**None**

Slicing a sprite sheet

1. Select the sheet: **Sprite Mode** = Multiple → Apply.
2. **Sprite Editor** → **Slice** → Type: **Grid By Cell Size** → your frame size.
3. **Slice** → **Apply**. The frames appear under the sheet in the Project window.

Your art on a prefab

1. Select the prefab's `Visual` child (never the root).
2. Drag your sprite into the SpriteRenderer's **Sprite**.
3. Animation without an Animator: `JamSpriteSwapper` on `Visual`: drag frames into **Idle**, **Run**, **Jump**, **Fall**, **Hurt** (top-down: **Walk Down**, **Walk Up**, **Walk Side**, drawn facing right). **Fps** sets the speed.

Keep your art in `Assets/_Game/Art`. Name sprites `spr_name_action_##`.

3D model import basics

- **Scale:** check the model next to the kit's player capsule; fix it with the model's **Scale Factor**.
- **Characters:** Rig → Animation Type = **Humanoid** (Setup Mixamo Character does this for Mixamo files).
- **Materials:** must use **URP Lit**, or they render pink. Extract materials from the model and set their shader to Universal Render Pipeline/Lit.
- **Colliders:** anything you stand on needs one (Box Collider is cheapest).
- `JamKit → 3D → Fit Controller To Model` resizes the player's CharacterController to a new character.

Sound effects from any event

1. Find the event (e.g. `OnJump` on the player, `OnCollected` on a coin, `OnEnter` on a Trigger Zone).
2. **+** → drag in an object with `JamAudio` (the player's `FX` has one).
3. Pick `JamAudio → PlayOneShot` → drop your sound in.

Many components also have a sound slot: **Pickup Sound**, **Hit Sound**, **Open Sound**, **Hurt Sound**, **Reach Sound**... `JamAudio` **Random Pitch** keeps repeats from sounding robotic.

Music

`MusicPlayer` (`JamMusicPlayer`) under `---- SYSTEMS ----`: drop a loop into **Clip**; keep **Volume** below your sound effects. Same clip in the next level = it keeps playing without restarting.

Parallax background

`JamParallax` on a background layer: **Parallax Factor** 0 = moves with the world, 1 = glued to the camera; 0.2–0.8 for layers in between. **Infinite Horizontal** repeats it sideways.

11 Troubleshooting

Problem	Fix
2D player falls through the floor	The ground needs a Collider 2D. Paint on `Tilemap_Ground` (it has Tilemap Collider 2D + Composite Collider 2D), or add them to your own tilemap.
3D player falls through the floor	The ground mesh needs a Collider (Box or Mesh Collider).
Can't jump	You're standing on something without a collider, or `maxJumps` = 0 in the `PlayerConfig` (`PlayerConfig_TopDown` has no jumping).
Pink sprites or materials	Use the template project (URP). 3D materials must be URP Lit, or run Setup Mixamo Character again.
Keyboard does nothing	Click inside the Game view first. Still nothing: Project Settings → Player → Active Input Handling = Both (restart the editor).
TMP pop-up appears	Click "Import TMP Essentials".
"Input Manager is marked for deprecation" notice	Harmless with Active Input Handling = Both: ignore it.
3D character stuck in T-pose	Rig isn't Humanoid or the Animator controller is missing: run Setup Mixamo Character.
3D character slides forward / snaps back	The Mixamo clip wasn't downloaded In Place: download it again with In Place ticked.
Camera goes through walls	Walls need colliders; the camera only pulls in against colliders.
Mouse cursor disappeared	It's locked while playing 3D: press Esc.
Win screen never shows	No `JamGameManager` in the scene: `JamKit → Validate Scene` → Fix.
Goal doesn't win	It's locked: check **Required Item Id** / **Required Amount** on `JamGoal` (the hint shows have / need).
Door doesn't open	The zone's **Required Item Id** must match the key's **Item Id**, and **On Enter** must call `JamDoor → Open`.
Coin counter missing on the HUD	`JamHUD` **Item Id** must match the collectibles' **Item Id** (default `coin`).
Next button missing / next level doesn't load	Set **Next Level Name** on the GameManager (exact scene name); the scene must be in Build Settings: use `JamKit → New Level` or Validate Scene → Fix.
Blank page on itch.io	Run `JamKit → Prepare WebGL Build` again (compression Disabled), zip the build folder's **contents** (not the folder) and re-upload.

Still stuck? `JamKit → Validate Scene` catches most problems. Field-by-field details: `Docs/JamKit_Reference.md`.

12 Submission checklist



Before you submit

- [ ] First build on the **first night of the jam**; a working build by the **second night**.
- [ ] `JamKit → Validate Scene`: all ✅ in every level.
- [ ] `MainMenu` is scene 0 in Build Settings; all levels added.
- [ ] Title, Win and Lose text changed to your game's.
- [ ] `JamKit → Prepare WebGL Build`: team and game name set.
- [ ] itch.io: Kind = **HTML**; zip of the build folder's **contents**; "This file will be played in the browser"; viewport **960 × 540**; **Fullscreen button**.
- [ ] Test the itch link in an **incognito window**.
- [ ] Cover image uploaded.
- [ ] Controls listed on the page.
- [ ] Credits: JamKit, CC0 art and sound (Kenney), Mixamo if used.
- [ ] Upload with a buffer before the deadline.

5 survival rules

1. **Make a build on the first night of the jam.**
   An ugly build that works beats a beautiful project that doesn't export.
2. **Win and lose first, content second.**
   A game isn't a game until it can end.
3. **Back up every few hours.**
   Zip the project folder (or use GitHub Desktop / Unity Version Control). One person merges scenes; others work in their own scene.
4. **Feature freeze on the morning of the last day.**
   The rest of the day is for bugs, polish and the itch page.
5. **Playtest with someone who's never seen it.**
   If they don't know what to do, add a sign (`JamDialogueTrigger`).

Credit line for your game page

```
Made with JamKit by Shounak Sobahani (MIT) — github.com/shounak00/JamKit
```

Licences: JamKit code MIT. Art and sound CC0 by Kenney (Pixel Platformer, Tiny Dungeon, Digital Audio, Impact Sounds, Interface Sounds, Music Jingles) and JamKit's own CC0 placeholders. No Mixamo files ship with the kit; if your game uses Mixamo, credit "Character/animations from Mixamo (Adobe)" and never redistribute the raw files.

Good luck at the jam.

Drag a prefab. Set its fields. Press Play.





